HLQ-OS: ISO Building & Compilation Guide

QUICK REFERENCE & PROJECT CONFIGURATION WORKFLOW

1. The Direct Command to Build Your ISO

To compile your assembly files, compile your Rust kernel, link them together, and produce the final bootable ISO file, simply run the following standard command in your terminal folder:

```
# This single command triggers the full build pipeline
make iso
```

2. Other Quick Target Commands

Your workspace supports several helpful shortcuts built into your Makefile execution routine:

- `make all` — Compiles the code and produces the kernel binary (`.bin`), but does not wrap it inside an ISO.
- `make run` — Builds everything, compiles the ISO, and boots your operating system immediately inside the QEMU emulator.
- `make clean` — Deletes the compiled artifacts directory (`build/`) and targets to force a completely fresh rebuild.

3. Complete Complete Makefile

Below is your complete, optimized Makefile structure with Rust kernel integration mapped accurately to cross-compile cleanly into the 64-bit target environment:

```
arch ?= x86_64
kernel := build/kernel-$(arch).bin
iso := build/hlq-os-$(arch).iso

# Complete path to the bare-metal cross-compiled target architecture static library
rust_os := target/x86_64-unknown-none/debug/libHLQ_OS.a

linker_script := src/arch/$(arch)/linker.ld
grub_cfg := src/arch/$(arch)/grub.cfg
assembly_source_files := $(wildcard src/arch/$(arch)/*.asm)
assembly_object_files := $(patsubst src/arch/$(arch)/%.asm, \
    build/arch/$(arch)/%.o, $(assembly_source_files))

.PHONY: all clean run iso kernel cargo

all: $(kernel)
```

```
clean:
        @rm -rf build
        @cargo clean

run: $(iso)
        @qemu-system-x86_64 -cdrom $(iso)

iso: $(iso)

$(iso): $(kernel) $(grub_cfg)
        @mkdir -p build/isofiles/boot/grub
        @cp $(kernel) build/isofiles/boot/kernel.bin
        @cp $(grub_cfg) build/isofiles/boot/grub
        @grub-mkrescue -o $(iso) build/isofiles 2> /dev/null
        @rm -rf build/isofiles

cargo:
        @cargo build --target x86_64-unknown-none

$(kernel): cargo $(assembly_object_files) $(linker_script)
        @ld -n -T $(linker_script) -o $(kernel) $(assembly_object_files) $(rust_os)

build/arch/$(arch)/%.o: src/arch/$(arch)/%.asm
        @mkdir -p $(shell dirname $@)
        @nasm -felf64 $< -o $@
```

> **Required Environment Tooling:** Ensure your underlying environment has the necessary bare-metal toolchain targets added via Rustup before running the make command:
> `rustup target add x86_64-unknown-none`